Laboratory Manual
Of
ARTIFICIAL INTELLIGENCE

B.Tech. III Year II Sem (CSE)

DEPARTMENT OF CSE

KESHAV MEMORIAL COLLEGE OF ENGINEERING
Chinthapallyguda(V), Ibrahimpatnam(M) Ranga Reddy Dist.

CS605PC: ARTIFICIAL INTELLIGENCE LAB

B.Tech. III Year II Sem.

L	T	P	C
0	0	2	1

Course Objectives:

- Become familiar with basic principles of AI toward problem solving, knowledge representation, and learning.

Course Outcomes:

- Apply basic principles of AI in solutions that require problem solving, knowledge representation, and learning.

LIST OF EXPERIMENTS

Write a Program to Implement the following using Python.

1. Breadth First Search
2. Depth First Search
3. Tic-Tac-Toe game
4. 8-Puzzle problem
5. Water-Jug problem
6. Travelling Salesman Problem
7. Tower of Hanoi
8. Monkey Banana Problem
9. Alpha-Beta Pruning
10. 8-Queens Problem

TEXT BOOKS:

1. Artificial Intelligence a Modern Approach, Third Edition, Stuart Russell and Peter Norvig, Pearson Education.

REFERENCE BOOKS:

1. Artificial Intelligence, 3rd Edn, E. Rich and K. Knight (TMH)
2. Artificial Intelligence, 3rd Edn., Patrick Henny Winston, Pearson Education.
3. Artificial Intelligence, Shivani Goel, Pearson Education.

1. Write a Program to Implement Breadth First Search problem using Python

```
def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            queue.extend(graph[node] - visited)
graph = {
    'A': {'B', 'C'},
    'B': {'A', 'D', 'E'},
    'C': {'A', 'F', 'G'},
    'D': {'B'},
    'E': {'B', 'H'},
    'F': {'C'},
    'G': {'C'},
    'H': {'E'}
}
bfs(graph, 'A')
```

<u>OUTPUT:</u>

A B C D E F G H

2. Write a Program to Implement Depth First Search problem using Python

```
def dfs_recursive(graph, node, visited=None):
    if visited is None:
        visited = set()

    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbor in graph[node]:
            dfs_recursive(graph, neighbor, visited)
graph = {
    'A': {'B', 'C'},
    'B': {'A', 'D', 'E'},
    'C': {'A', 'F', 'G'},
    'D': {'B'},
    'E': {'B', 'H'},
    'F': {'C'},
    'G': {'C'},
    'H': {'E'}
}
dfs_recursive(graph, 'A')
```

<u>OUTPUT:</u>

A B D E H C F G

3. Write a Program to Implement Tic-Tac-Toe game problem using Python

```
def print_board(board):
    for row in board:
        print(" | ".join(row))
        print("-" * 9)

def check_winner(board, player):
    # Check rows, columns, and diagonals
    for i in range(3):
        if all(cell == player for cell in board[i]) or \
           all(board[j][i] == player for j in range(3)):
            return True
    if all(board[i][i] == player for i in range(3)) or \
       all(board[i][2 - i] == player for i in range(3)):
        return True
    return False

def is_draw(board):
    return all(cell in ['X', 'O'] for row in board for cell in row)

def tic_tac_toe():
    board = [["1", "2", "3"],
             ["4", "5", "6"],
             ["7", "8", "9"]]

    current_player = "X"

    while True:
        print_board(board)
```

```
        move = input(f"Player {current_player}, enter your move (1-9): ")

        if not move.isdigit() or not (1 <= int(move) <= 9):
            print("Invalid input. Try again.")
            continue

        move = int(move) - 1
        row, col = divmod(move, 3)

        if board[row][col] in ["X", "O"]:
            print("Spot already taken. Try again.")
            continue

        board[row][col] = current_player

        if check_winner(board, current_player):
            print_board(board)
            print(f"□ Player {current_player} wins!")
            break

        if is_draw(board):
            print_board(board)
            print("It's a draw!")
            break

        current_player = "O" if current_player == "X" else "X"

# Start the game
tic_tac_toe()
```

Output

```
1 | 2 | 3
---------
4 | 5 | 6
---------
7 | 8 | 9
---------
X | 2 | 3
---------
4 | 5 | 6
---------
7 | 8 | 9
---------
X | 2 | 3
---------
4 | O | 6
---------
7 | 8 | 9
---------
X | X | 3
---------
4 | O | 6
---------
7 | 8 | 9
---------
Player O, enter your move (1-9):
```

4. Write a Program to Implement 8-Puzzle problem using Python

```
import heapq

# Manhattan Distance Heuristic
def manhattan_distance(state, goal):
    distance = 0
    for i in range(3):
        for j in range(3):
            val = state[i][j]
            if val != 0:
                goal_x, goal_y = [(ix, iy) for ix, row in enumerate(goal)
                                  for iy, v in enumerate(row) if v == val][0]
                distance += abs(goal_x - i) + abs(goal_y - j)
    return distance

# Find the (x, y) position of 0 (blank tile)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate neighboring states
def get_neighbors(state):
    x, y = find_blank(state)
    moves = [(-1,0), (1,0), (0,-1), (0,1)]
    neighbors = []
    for dx, dy in moves:
        nx, ny = x+dx, y+dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]
```

```
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(tuple(tuple(row) for row in new_state))
    return neighbors

# A* Search Algorithm
def a_star(start, goal):
    open_set = []
    heapq.heappush(open_set, (0 + manhattan_distance(start, goal), 0, start, []))
    visited = set()

    while open_set:
        f, g, current, path = heapq.heappop(open_set)
        if current in visited:
            continue
        visited.add(current)

        if current == goal:
            return path + [current]

        for neighbor in get_neighbors(current):
            if neighbor not in visited:
                new_g = g + 1
                new_f = new_g + manhattan_distance(neighbor, goal)
                heapq.heappush(open_set, (new_f, new_g, neighbor, path + [current]))

    return None

# Helper to print state
def print_state(state):
    for row in state:
        print(" ".join(str(n) if n != 0 else " " for n in row))
```

```
    print("---------")

# Example Usage
if __name__ == "__main__":
    start_state = (
        (1, 2, 3),
        (4, 0, 6),
        (7, 5, 8)
    )

    goal_state = (
        (1, 2, 3),
        (4, 5, 6),
        (7, 8, 0)
    )

    solution = a_star(start_state, goal_state)

    if solution:
        print("✅Solution found in", len(solution) - 1, "moves:\n")
        for step in solution:
            print_state(step)
    else:
        print("❌No solution found.")
```

Output

```
1 2 3
4   6
7 5 8
---------
```

```
1 2 3
4 5 6
7   8
---------
1 2 3
4 5 6
7 8
---------
```

5. Write a Program to Implement Water-Jug problem using Python

```
from collections import deque

def is_goal(state, target):
    return target in state

def get_next_states(state, capacity1, capacity2):
    x, y = state
    return [
        (capacity1, y),          # Fill Jug1
        (x, capacity2),          # Fill Jug2
        (0, y),                  # Empty Jug1
        (x, 0),                  # Empty Jug2
        (0, x + y) if x + y <= capacity2 else (x - (capacity2 - y), capacity2),  # Pour Jug1 ->
Jug2
        (x + y, 0) if x + y <= capacity1 else (capacity1, y - (capacity1 - x))   # Pour Jug2 ->
Jug1
    ]

def water_jug_bfs(capacity1, capacity2, target):
    visited = set()
    queue = deque()
    queue.append(((0, 0), []))

    while queue:
        state, path = queue.popleft()
        if state in visited:
            continue
        visited.add(state)
```

```
        if is_goal(state, target):
            return path + [state]

        for next_state in get_next_states(state, capacity1, capacity2):
            if next_state not in visited:
                queue.append((next_state, path + [state]))

    return None

# Example usage
jug1 = 4
jug2 = 3
target = 2

solution = water_jug_bfs(jug1, jug2, target)

if solution:
    print(f"✅Solution found in {len(solution)-1} steps:")
    for step in solution:
        print(f"Jug1: {step[0]}L, Jug2: {step[1]}L")
else:
    print("❌No solution found.")
```

OUTPUT

```
Jug1: 0L, Jug2: 0L
Jug1: 0L, Jug2: 3L
Jug1: 3L, Jug2: 0L
Jug1: 3L, Jug2: 3L
Jug1: 4L, Jug2: 2L
```

6. Write a Program to Implement Travelling Salesman Problem using Python

```
import itertools


def tsp_brute_force(graph, start):
    n = len(graph)
    min_path = float('inf')
    best_route = []

    cities = list(range(n))
    cities.remove(start)
    for perm in itertools.permutations(cities):
        current_cost = 0
        k = start
        for j in perm:
            current_cost += graph[k][j]
            k = j
        current_cost += graph[k][start]

        if current_cost < min_path:
            min_path = current_cost
            best_route = [start] + list(perm) + [start]
    return best_route, min_path
# Input Graph
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
```

```
route, cost = tsp_brute_force(graph, 0)
print("Best Route:", route)
print("Minimum Cost:", cost)
```

<u>Output</u>

Best Route: [0, 1, 3, 2, 0]

Minimum Cost: 80

7. Write a Program to Implement Tower of Hanoi Problem using Python

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    tower_of_hanoi(n - 1, source, destination, auxiliary)
    print(f"Move disk {n} from {source} to {destination}")
    tower_of_hanoi(n - 1, auxiliary, source, destination)

# Example usage
num_disks = 3
tower_of_hanoi(num_disks, 'A', 'B', 'C')
```

Output

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

8. Write a Program to Implement Monkey Banana Problem using Python

```
# State = (Monkey Location, Box Location, Monkey on Box?, Has Banana?)
# Locations: "door", "middle", "window"

def is_goal(state):
    return state == ("middle", "middle", True, True)

def successors(state):
    monkey, box, on_box, has_banana = state
    result = []

    if not has_banana:
        # Monkey walks
        if not on_box:
            for loc in ["door", "middle", "window"]:
                if loc != monkey:
                    result.append((loc, box, on_box, has_banana))

        # Monkey pushes box
        if monkey == box and not on_box:
            for loc in ["door", "middle", "window"]:
                if loc != box:
                    result.append((loc, loc, on_box, has_banana))

        # Monkey climbs box
        if monkey == box and not on_box:
            result.append((monkey, box, True, has_banana))

        # Monkey grabs banana
```

```
        if monkey == "middle" and box == "middle" and on_box:
            result.append((monkey, box, on_box, True))

    return result

def monkey_banana_problem():
    initial_state = ("door", "window", False, False)
    frontier = [initial_state]
    explored = set()
    parent_map = {}

    while frontier:
        current = frontier.pop(0)
        explored.add(current)

        if is_goal(current):
            # Reconstruct path
            path = [current]
            while current in parent_map:
                current = parent_map[current]
                path.append(current)
            path.reverse()
            return path

        for child in successors(current):
            if child not in explored and child not in frontier:
                parent_map[child] = current
                frontier.append(child)

    return None
```

```
# Run the program
solution = monkey_banana_problem()


if solution:
    for step in solution:
        print(f"Monkey at {step[0]}, Box at {step[1]}, On box: {step[2]}, Has banana: {step[3]}")
else:
    print("No solution found.")
```

OUTPUT

```
Monkey at door, Box at window, On box: False, Has banana: False
Monkey at window, Box at window, On box: False, Has banana: False
Monkey at middle, Box at middle, On box: False, Has banana: False
Monkey at middle, Box at middle, On box: True, Has banana: False
Monkey at middle, Box at middle, On box: True, Has banana: True
```

9. Write a Program to Implement Alpha-Beta Pruning using Python

```
def alpha_beta_pruning(node, depth, alpha, beta, maximizing_player):
    if depth == 0 or isinstance(node, int):
        return node
    if maximizing_player:
        max_eval = float('-inf')
        for child in node:
            eval = alpha_beta_pruning(child, depth-1, alpha, beta, False)
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break  # β cut-off
        return max_eval
    else:
        min_eval = float('inf')
        for child in node:
            eval = alpha_beta_pruning(child, depth-1, alpha, beta, True)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break  # α cut-off
        return min_eval
        # Tree = [ [3, 5], [6, 9], [1, 2], [0, -1] ] → grouped as a 2-level tree
game_tree = [
    [3, 5],
    [6, 9],
    [1, 2],
    [0, -1]
]
result = alpha_beta_pruning(game_tree, depth=2, alpha=float('-inf'), beta=float('inf'),
```

```
maximizing_player=True)
print("Optimal value:", result)
```

<u>Output</u>

Optimal value: 6

10. Write a Program to Implement 8-Queens Problem using Python

```
N = 8
def print_solution(board):
    for row in board:
        print(" ".join("Q" if col else "." for col in row))
    print("\n")

def is_safe(board, row, col):
    # Check column
    for i in range(row):
        if board[i][col]:
            return False

    # Check upper-left diagonal
    for i, j in zip(range(row-1, -1, -1), range(col-1, -1, -1)):
        if board[i][j]:
            return False
    # Check upper-right diagonal
    for i, j in zip(range(row-1, -1, -1), range(col+1, N)):
        if board[i][j]:
            return False
    return True
def solve_n_queens(board, row=0):
    if row == N:
        print_solution(board)
        return

    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1
```

```
        solve_n_queens(board, row + 1)
        board[row][col] = 0  # Backtrack

# Create empty board
board = [[0]*N for _ in range(N)]
solve_n_queens(board)
```

Sample Output

```
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .

Q . . . . . . .
. . . . . Q . .
. . . . . . . Q
. . Q . . . . .
. . . . . . Q .
. . . Q . . . .
. Q . . . . . .
. . . . Q . . .

Q . . . . . . .
. . . . . . Q .
. . . Q . . . .
. . . . . Q . .
. . . . . . . Q
. Q . . . . . .
. . . . Q . . .
. . Q . . . . .
```